41. Containers and Multi-stage Docker Builds for Go

41. Containers and Multi-stage Docker Builds for Go

Phase: Phase 6 — Systems and infrastructure
Estimated time: 1–2 days
Slug: `41-containers-docker`

Learning objectives

- Write a multi-stage Dockerfile for Go
- Build static/small images
- Run as non-root
- Understand layer caching

Prerequisites

Complete prior numbered lessons. Type every example yourself.

Multi-stage pattern

```
FROM golang:1.22 AS build
WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 go build -o /out/app ./cmd/api

FROM gcr.io/distroless/static:nonroot
COPY --from=build /out/app /app
USER nonroot:nonroot
ENTRYPOINT ["/app"]
```

Why

- Tiny attack surface
- Reproducible builds
- Same artifact from laptop to prod

Pin base image digests in serious deployments. Pass version via ldflags.

Common mistakes (especially from Python instincts)

- Shipping build toolchain in prod image
- Running as root
- COPY . . before go mod download (breaks cache)

Exercises

Work in `exercises/41-containers-docker/`. Read that folder's `README.md`.

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/41-containers-docker/` until you have an honest failing attempt.

Self-check

1. Why multi-stage?
2. What does CGO_ENABLED=0 help?
3. Why nonroot?

Further reading

- https://docs.docker.com/language/golang/build-images/